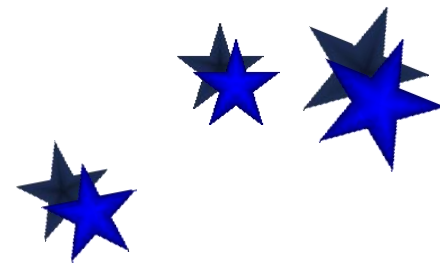


算法设计与分析

主讲教师：徐红艳

联系电话：**13840597032**



课程简介

◆ 计算机系统上的任何软件（系统软件、应用软件）都是按照特定的**算法**来实现的。

- 设计：求解问题的一系列步骤的设计，用来将输入数据转换成输出结果。



- 分析：算法**性能**的好坏直接决定所实现软件的优劣。



课程简介

近似 算法	随机 算法	处理难解问 题的策略	
NP 完全理论			
算法分析与问题的计算复杂性			
线性规划		网络流算法	
分治 策略	动态 规划	贪心 算法	回溯与 分支限界
数学基础、数据结构			

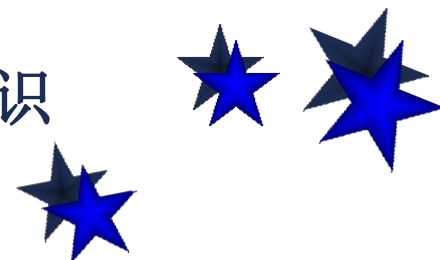
问题处理策略

计算复杂性理论

算法分析方法

算法设计技术

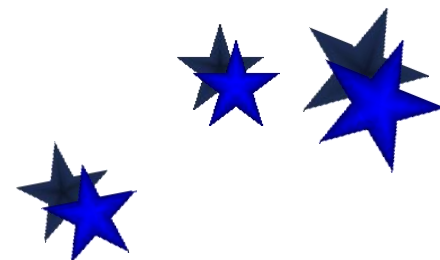
基础知识



课程简介

◆参考书目/MOOC资源

- (1) 屈婉玲. 算法设计与分析(第2版). 清华大学出版社.
- (2) 李春葆. 算法设计与分析(第2版). 清华大学出版社.
- (3) Anany Levitin著, 潘彦译. 算法设计与分析基础. 清华大学出版社.
- (4) 师智斌. 算法分析与设计及案例教程. 清华大学出版社.
- (5) 中国大学MOOC, 算法设计与分析, 屈婉玲.



第1章 基础知识

1.1 有关算法的基本概念

- 算法设计的任务——调度问题、任务分配
- 问题计算复杂度的界定——排序问题
- 计算复杂性理论——货郎问题、背包问题、双机调度
- 算法及其时间复杂度——线性检索

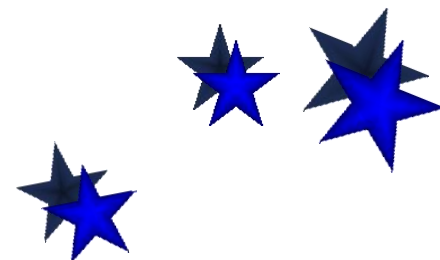
1.2 算法的伪码表示

1.3 数学基础知识

1.3.1 函数渐近的界

1.3.2 求和的方法

1.3.3 递推方程求解方法



1.1 有关算法的基本概念

■ 算法设计的任务

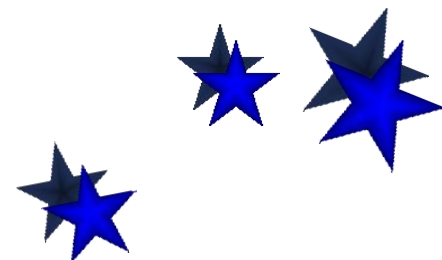
例1：调度问题

问题描述：有 n 项任务，每项任务加工时间已知。从0时刻开始陆续安排到一台机器上加工。每个任务的完成时间是从0时刻到该任务加工截止的时间。

求解：总完成时间（所有任务完成时间之和）最短的安排方案

实例：任务集 $S = \{1, 2, 3, 4, 5\}$,

加工时间: $t_1=3, t_2=8, t_3=5, t_4=10, t_5=15$



1.1 有关算法的基本概念

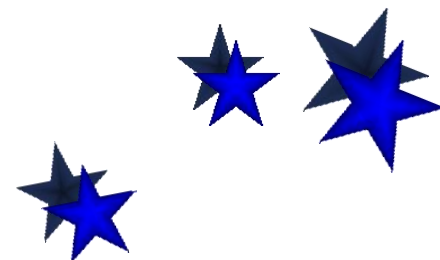
求解策略（算法）：**贪心策略**

按加工时间 (3,8,5,10,15) 从小到大安排，即按照1, 3, 2, 4, 5的顺序调度任务。



总完成时间：

$$\begin{aligned} t &= 3 + (3+5) + (3+5+8) + (3+5+8+10) + (3+5+8+10+15) \\ &= 3 \times 5 + 5 \times 4 + 8 \times 3 + 10 \times 2 + 15 \\ &= 94 \end{aligned}$$



1.1 有关算法的基本概念

问题建模:

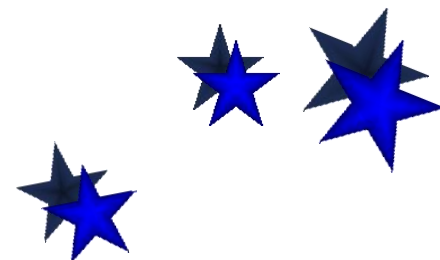
(1) **输入**: 任务集 $S=\{1, 2, \dots, n\}$, 第 j 项任务加工时间: $t_j, \in \mathbb{Z}^+, j=1, 2, \dots, n$

(2) **输出**: 调度策略 I , 应为 S 的排列 $i_1, i_2, \dots, i_n$

(3) 目标函数: I 的完成时间, $t(I) = \sum_{k=1}^n (n-k+1)t_{i_k}$

(4) 解: 设为 I^* , 使得 $t(I^*)$ 的值达到最小, 即

$$t(I^*) = \min \{ t(I) \mid I \text{ 为 } S \text{ 的排列} \}$$

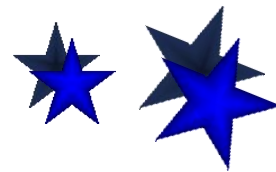


1.1 有关算法的基本概念

选择策略：贪心策略

- (1) 设计策略：加工时间短的先做
- (2) 设计算法：

```
long TaskArrange(TASK taskset[],int n)  
{ Sort(taskset); //按照时间成员进行排序  
sum=0; item=0;  
for(i=1;i<=n;i++)  
{ item=item+taskset.t[i];  
sum=sum+item;}  
return(sum);}
```



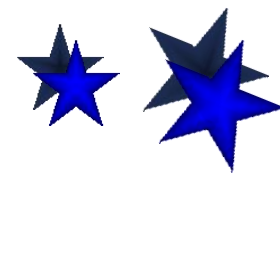
1.1 有关算法的基本概念

算法正确性验证：对所有输入实例都得到最优解

证：假如调度 f 第 i, j 项任务相邻且有逆序，
即 $t_i > t_j$ 。交换任务 i 和 j 得到调度 g ，



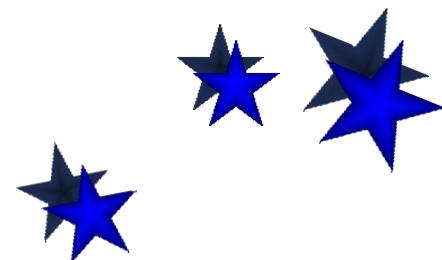
总完成时间 $t(g) - t(f) = t_j - t_i < 0$



1.1 有关算法的基本概念

例2：任务分配问题

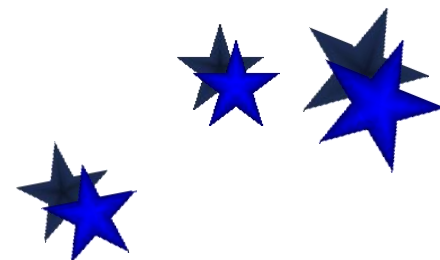
- (1) 问题描述： m 元钱，投资 n 个项目. 效益函数 $f_i(x)$ ，表示第 i 个项目投 x 元的效益， $i=1, 2, \dots, n$ 。
- (2) 求解： 如何分配每个项目的钱数使得总效益最大？
- (3) 实例： 5 万元，投资给 4 个项目，效益函数如下表所示：



1.1 有关算法的基本概念

投资金额 $m=5$ ， 项目数 $n=4$

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24



1.1 有关算法的基本概念

问题建模:

(1) 输入: $n, m, f_i(x), i=1,2, \dots, n, x=1,2, \dots, m$

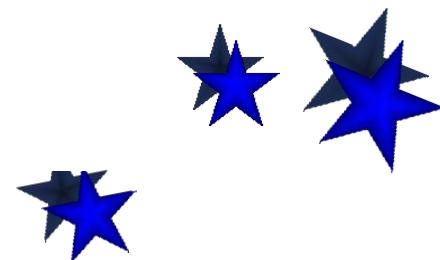
(2) 输出: n 维向量 $\langle x_1, x_2, \dots, x_n \rangle$, x_i 是第 i 个项目分到的钱数, 使得下述条件满足:

目标函数

$$\max \sum_{i=1}^n f_i(x_i),$$

约束条件

$$\sum_{i=1}^n x_i = m, \quad x_i \in \mathbb{N}$$



1.1 有关算法的基本概念

选择策略：蛮力策略（穷举策略）

- （1）设计策略：从所有可行策略中选择出效益最大的方案。
- （2）设计算法

```
status MaxBonus(int m,int n,int Bonus[m+1][n])  
{ for(i=0;i<=m;i++)  
    for(j=0;j<=n-i;j++)  
        for(k=0;k<=n-i-j;k++)  
            { l=n-i-j-k;  
                f=Bonus[i][0]+Bonus[j][1]+Bonus[k][2]+Bonus[l][3];  
                if(maxf<f){ maxf=f; t[]={i,j,k,l};  
            }  
    return(maxf,t); }
```

1.1 有关算法的基本概念

实例计算

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

$$x_1 + x_2 + x_3 + x_4 = 5$$

$$s_1 = \langle 0, 0, 0, 5 \rangle, v(s_1) = 24$$

$$s_2 = \langle 0, 0, 1, 4 \rangle, v(s_2) = 25$$

$$s_3 = \langle 0, 0, 2, 3 \rangle, v(s_3) = 32$$

...

$$s_{56} = \langle 5, 0, 0, 0 \rangle, v(s_{56}) = 15$$

解: $s = \langle 1, 0, 3, 1 \rangle$,

最大效益: $11 + 30 + 20 = 61$



1.1 有关算法的基本概念

算法正确性验证：因为是从所有的可行解中找出的最佳方案，因此所得的解一定是最优的。

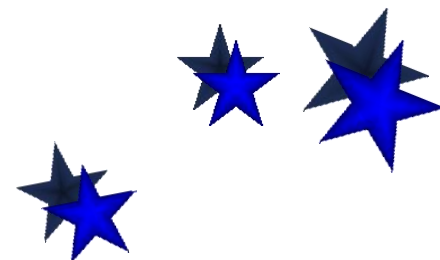
算法分析：

（1）效率分析：循环的层数与任务数 n 相关，所以当 m 、 n 规模很大时，算法的效率是非常低下的。

（2）实现可行性分析：因没有控制循环层数的机制，因此算法的实现很困难。

思考：有没有更好的实现策略？

动态规划策略



1.1 有关算法的基本概念

小结：通过上述两个例子的分析，我们可知**算法设计的任务**是：

- （1）**建模**：对输入参数和解给出形式化或半形式化的描述，分析问题的目标函数、约束函数等。
- （2）**设计算法**：选择**恰当的算法策略**进行设计
- （3）**分析算法的正确性**：是否对所有的实例都得到正确的解（尤其贪心策略一定要证明）
- （4）**分析算法效率**：是否为指数级的（NP难问题），若是，能否为其找到近似多项式阶的解。



1.1 有关算法的基本概念

何谓“恰当的算法策略”

例3：硬币找零问题

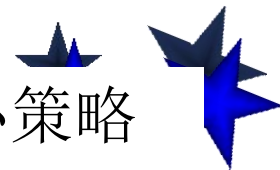
(1) 情形1：若某国的币种如下：100、50、20、10、5、2、1，请给出发放140元的最少张数。

使用贪心策略实现：

- (1) 将所有的币种升序排列。
- (2) 尽可能多的选择币种最大的构成问题的解。

问题的解：1张100+2张20=3张

可以证明，对于上述币种，任何给定的数值使用贪心策略均能找到最优解。



1.1 有关算法的基本概念

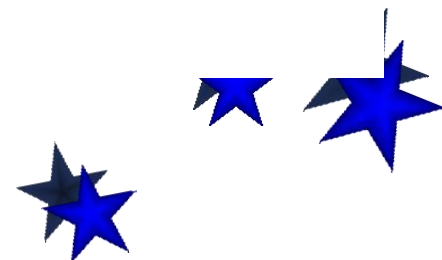
(2) 情形2：若某国有下列面值硬币 (25,10,1)，请给出找回30分对应的最少硬币方案：

使用贪心策略实现：1枚25分，5枚1分，**合计6枚**

问题的最优解：3枚10分，**合计3枚**

情形2的求解策略：动态规划策略

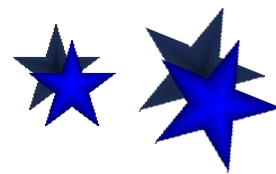
结论：为解决的问题选择**恰当的算法策略**。



1.1 有关算法的基本概念

- 问题计算复杂度的界定：排序问题
 - 数据结构中研究的排序方法

算法	最坏情况下	平均情况下
插入排序	$O(n^2)$	$O(n^2)$
冒泡排序	$O(n^2)$	$O(n^2)$
快速排序	$O(n^2)$	$O(n\log n)$
堆排序	$O(n\log n)$	$O(n\log n)$
二分归并排序	$O(n\log n)$	$O(n\log n)$



1.1 有关算法的基本概念

- 例4：根据实例分析每种排序方法的实现过程

输入

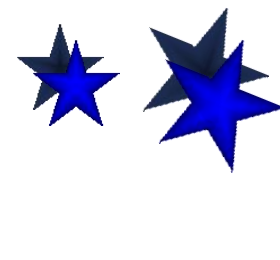
5	7	1	3	6	2	4
---	---	---	---	---	---	---

排序 →

- 插入排序
- 冒泡排序
- 快速排序
- 堆排序
- 二分归并排序

输出

1	2	3	4	5	6	7
---	---	---	---	---	---	---



1.1 有关算法的基本概念

- 问题计算复杂度的界定：解决同一类问题采用的不同策略进行效率分析。

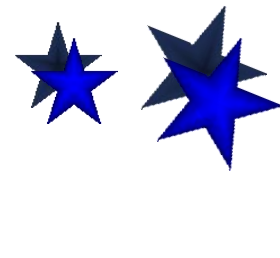
针对排序问题：

哪个排序算法效率最高？

n^2 插入排序
冒泡排序
快速排序

$n \log n$ 堆排序
归并排序

是否可找到更好的、效率更高的排序算法？



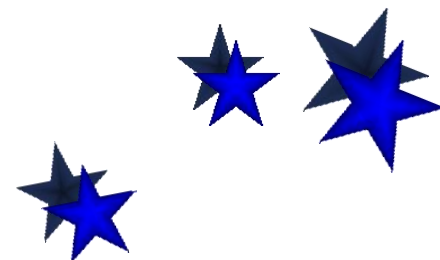
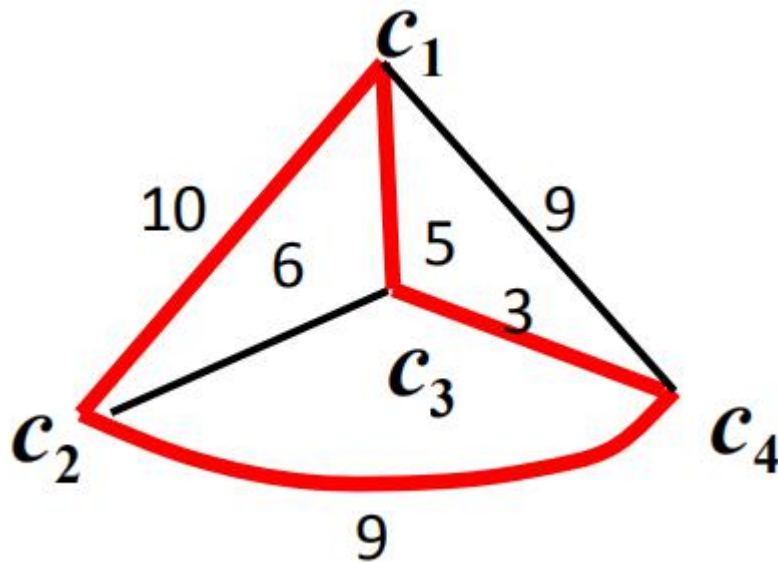
1.1 有关算法的基本概念

■ 计算复杂性理论

例题5：NP难问题1：货郎问题

(1) 问题描述：有 n 个城市，已知任两个城市之间的距离（图的存储：邻接表、邻接矩阵）。

(2) 求一条每个城市恰好经过1次的回路，使得总长度最小。



1.1 有关算法的基本概念

问题建模

排列问题，复杂度为： $n!$

- 输入

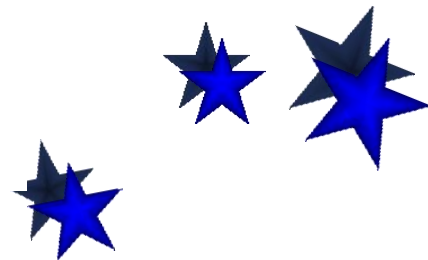
有穷个城市的集合 $C = \{c_1, c_2, \dots, c_n\}$,

距离 $d(c_i, c_j) = d(c_j, c_i) \in \mathbb{Z}^+$, $1 \leq i < j \leq n$

- 解： $1, 2, \dots, n$ 的排列 $k_1, k_2, \dots, k_n$ 使得：

$$\min \left\{ \sum_{i=1}^{n-1} d(c_{k_i}, c_{k_{i+1}}) + d(c_{k_n}, c_{k_1}) \right\}$$

- 现状：至今没找到有效的算法



1.1 有关算法的基本概念

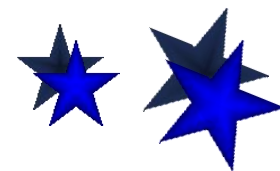
例题6: NP难问题2: 0-1背包问题

(1) 问题描述: 有 n 个件物品要装入背包, 第 i 件物品的重量 w_i , 价值 v_i , $i=1,2,\dots,n$ 。背包最多允许装入的重量为 B 。

(2) 求解: 如何选择装入背包的物品, 使得总价值达到最大

实例: $n=4$, $B=6$, 物品的重量和价值如下

标号	1	2	3	4
重量 w_i	3	4	5	2
价值 v_i	7	9	9	2



1.1 有关算法的基本概念

问题建模

子集问题，复杂度为： 2^n

(1) 输入： n 个物品的重量 w_i 与价值 v_i

(2) 问题的解： $0-1$ 向量 $\langle x_1, x_2, \dots, x_n \rangle$
 $x_i=1 \Leftrightarrow$ 物品 i 装入背包

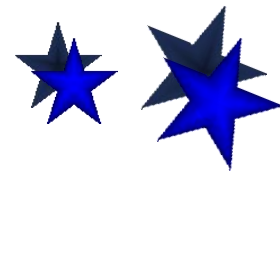
目标函数

$$\max \sum_{i=1}^n v_i x_i$$

约束条件

$$\sum_{i=1}^n w_i x_i \leq B$$

$$x_i = 0, 1, i = 1, 2, \dots, n$$

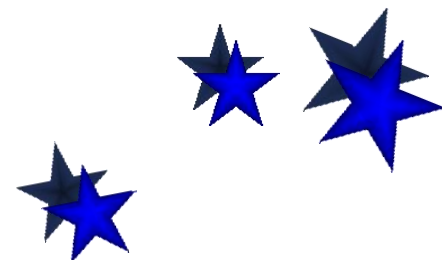


1.1 有关算法的基本概念

例题7：NP难问题3：双机调度问题

(1) 问题描述：有 n 项任务, 任务 i 的加工时间为 t_i , $t_i \in \mathbb{Z}^+$, $i=1,2,\dots,n$ 。用两台相同的机器加工, 从0时刻开始计时, 完成时间是后停止加工机器的停机时间。

(2) 求解：问如何把这些任务分配到两台机器上, 使得完成时间达到最小?



1.1 有关算法的基本概念

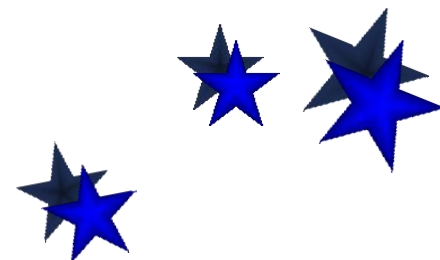
实例：任务集 $S = \{1, 2, 3, 4, 5, 6\}$

$$t_1=3, t_2=10, t_3=6, t_4=2, t_5=1, t_6=7$$

解：机器 1 的任务：1, 2, 4

机器 2 的任务：3, 5, 6

完成时间： $\max\{3+10+2, 6+1+7\}=15$



1.1 有关算法的基本概念

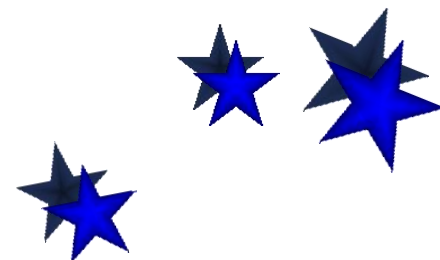
问题建模

解: 0-1向量 $\langle x_1, x_2, \dots, x_n \rangle$, $x_i=1$ 表示任务 i 分配到第一台机器, $i=1, 2, \dots, n$.

不妨设机器1的加工时间 $\leq$ 机器2的加工时间令 $T=t_1+t_2+\dots+t_n$, $D=\lfloor T/2 \rfloor$, 机器1的加工时间不超过 D , 且达到最大.



思考: 目标函数与约束条件是什么?



1.1 有关算法的基本概念

- **NP-hard问题**

- (1) 这样的问题有数千个，大量存在于各个应用领域
- (2) 至今没找到有效算法：现有的算法的运行时间是输入规模的指数或更高阶函数。
- (3) 至今没有人能够证明对于这类问题不存在多项式时间的算法。
- (4) 从是否存在多项式时间算法的角度看，这数千个NP难问题彼此是等价的。若一个NP难问题存在多项式时间，则这些问题的难度处于可有效计算的边界。



1.1 有关算法的基本概念

Algorithm + Data Structure = **Programming**

- 好的算法

- (1) 提高求解问题的效率

- (2) 节省存储空间

- 算法的研究目标

- (1) 实际问题→建模并寻找算法

- (2) 设计的算法→算法的评价

- (3) 算法类（排序）→问题复杂度估计

- (4) 问题类(NP难)→能够求解的边界

算法设计技术

算法分析方法

问题复杂度分析

计算复杂性理论



1.1 有关算法的基本概念

■ 算法及时间复杂度估算

➤ 算法中涉及的基本概念

(1) 问题：需要回答的一般性提问，通常含若干参数。

(2) 问题描述

- 定义问题参数(集合,变量,函数,序列等)
- 说明每个参数的取值范围及参数间的关系
- 定义问题的解
- 说明解满足的条件(优化目标或约束条件)

(3) 问题实例：参数的一组赋值可得到问题的一个实例。

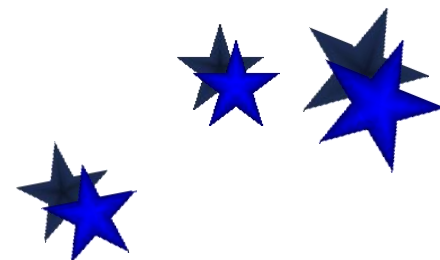


1.1 有关算法的基本概念

(4) 算法：有限条指令的序列，这个指令序列确定了解决某个问题的一系列运算或操作。

(5) 算法 A 解决问题 P

- 把问题 P 的**任何实例**作为算法 A 的输入
- 每步计算是确定性的
- A 能够在有限步停机
- 输出该实例的正确的解



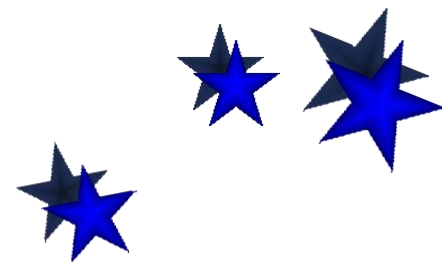
1.1 有关算法的基本概念

➤ 算法时间复杂度估算

(1) 算法时间复杂度: 针对指定**基本运算**, 计数算法所做该运算的次数, 通常是**问题输入规模**的函数。

(2) 输入规模: 输入串编码长度。例如:

- 排序: 数组中元素个数 n
- 检索: 被检索数组的元素个数 n
- 整数乘法: 两个整数的位数 m, n
- 矩阵相乘: 矩阵的行列数 i, j, k
- 图的遍历: 图的顶点数 n , 边数 m



1.1 有关算法的基本概念

(3) 基本运算：比较、加法、乘法、置指针, 交换, 赋值...
例如：

- 排序: 元素之间的比较
- 检索: 被检索元素 x 与数组元素的比较
- 整数乘法: 每位数字相乘(位乘) 1 次 m 位和 n 位整数相乘要做 mn 次位乘
- 矩阵相乘: 每对元素乘 1 次 $i \times j$ 矩阵与 $j \times k$ 矩阵相乘要做 ijk 次乘法
- 图的遍历: 置指针

(4) 算法基本运算次数可表为输入规模的函数。

(5) 给定问题和基本运算就决定了一个算法类。



1.1 有关算法的基本概念

(6) 算法时间复杂度估算方法

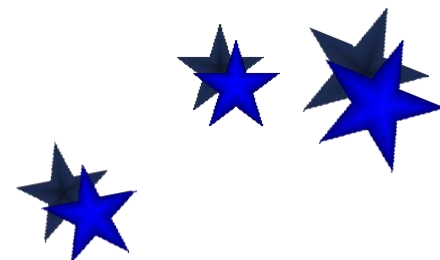
- 方法1：相同输入规模的不同实例，算法的基本运算次数也不一样，可分析如下三种时间复杂度：

- *最好时间复杂度

- *最坏时间复杂度

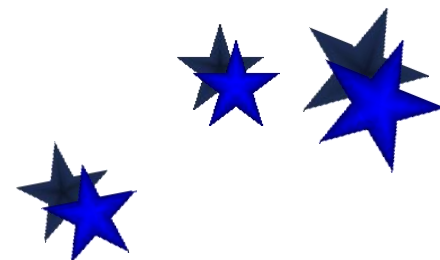
- *平均时间复杂度

- 方法2：渐进时间复杂度（1.3节）



1.1 有关算法的基本概念

- 设一个算法的输入规模为 n ， D_n 是所有输入的集合，任一输入 $I \in D_n$ ， $P(I)$ 是 I 出现的概率，有 $\sum P(I) = 1$ ， $T(I)$ 是算法在输入 I 下所执行的基本语句次数，则：
 - (1) 算法的最好情况为： $G(n) = \min_{I \in D_n} \{T(I)\}$ ，是指算法在所有输入 I 下所执行基本语句的最少次数。
 - (2) 算法的最坏情况为： $W(n) = \max_{I \in D_n} \{T(I)\}$ ，是指算法在所有输入 I 下所执行基本语句的最大次数。

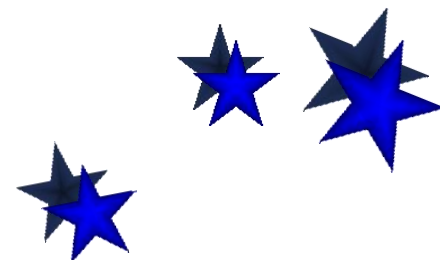


1.1 有关算法的基本概念

(3) 该算法的平均执行时间为

$$A(n) = \sum_{I \in D_n} P(I) * T(I)$$

- * 也就是说算法的平均情况是指用各种可能输入下的基本语句执行次数的带权平均值。
- * 必须知道所有输入实例出现的概率，即所有输入的分布情况。通常假定输入是平均分布的（等概率）。



1.1 有关算法的基本概念

- 最好、最坏、平均时间复杂度举例

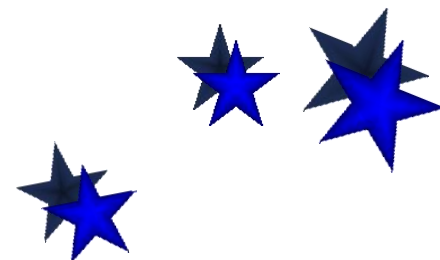
例题8：检索问题——线性检索法

输入：非降顺序排列的数组 L , 元素个数 n , 待检索数 x

输出：位置 i ，若 x 在 L 中, i 是 x 首次出现的下标；

否则 $i = 0$

基本运算： x 与 L 中元素的比较运算



1.1 有关算法的基本概念

- 实现算法1

$i=1$, 将 x 与 $L[i]$ 比较. 如果 $x=L[i]$, 则算法停止, 输出 i ; 如果不等, 则把 i 加1, 继续 x 与 $L[i]$ 的比较, 如果 $i>n$, 则停机并输出0.

实例

1

2

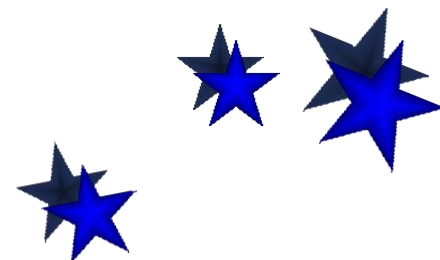
3

4

5

$x = 4$, 需要比较 4 次

$x = 2.5$, 需要比较 5 次



1.1 有关算法的基本概念

```
int Find1(double L[],int n,double x)  
{ int i=1;  
  while (i<=n)  
  { if (L[i]==x) break;  
    i++;  
  }  
  if (i<=n) return i;  
  else return 0;  
}
```



1.1 有关算法的基本概念

(1) 分析该算法在等概率情况下**成功查找**到值为 x 的元素的

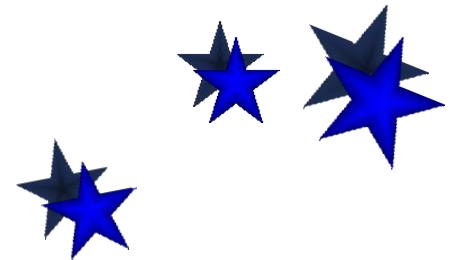
✓ 最好时间复杂度

第一个元素 $L[1]$ 等于 x ，此时基本语句仅执行一次，此时呈现**最好的情况**，即 $G(n)=O(1)$

✓ 最坏时间复杂度

当 L 中最后一个元素 $L[n-1]$ 等于 x ，此时基本语句执行 n 次，此时呈现**最坏的情况**，即 $W(n)=O(n)$ 。

✓ 平均时间复杂度



1.1 有关算法的基本概念

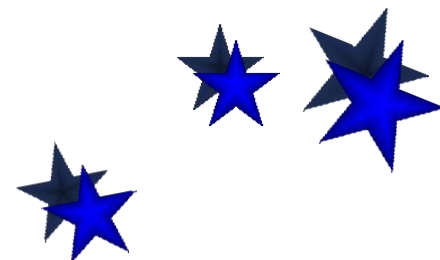
✓ 平均时间复杂度

实例

1	2	3	4	5
---	---	---	---	---

对于其他情况，假设查找每个元素的概率相同，则 $P(L[i])=1/n$
($1 \leq i \leq n$)，而成功找到 $L[i]$ 元素时基本语句正好执行 i 次，所以
:

$$A(n) = \sum_{i=1}^n \frac{1}{n} (i+1) = \frac{1}{n} \sum_{i=1}^n (i+1) = \frac{n+1}{2} = O(n)。$$



1.1 有关算法的基本概念

(2) 该问题可设计多少个输入实例进行复杂度分析？

实例

1

2

3

4

5

查找成功： n 个实例，比较次数为 i

$$x = L[1], x = L[2], \dots, x = L[n]$$

查找不成功： $n+1$ 个实例，比较次数均为 n

$$x < L[1], L[1] < x < L[2], L[2] < x < L[3], \dots, L[n] < x$$

最坏情况下时间： $W(n) = n$

最坏的输入： x 不在 L 中或 $x = L[n]$ ，要做 n 次比较



1.1 有关算法的基本概念

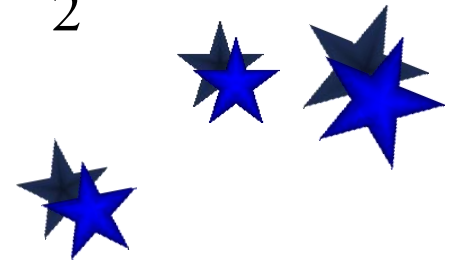
(3) 假设被查值 x 在数组 L 中的概率是 q ，求算法的时间复杂度。

- 对于成功查找，等概率情况下元素 $L[i]$ 被查找到的概率 $P(L[i])=q/n$ ，成功找到 $L[i]$ 元素时基本操作正好执行 i 次。
- 对于不成功查找，其概率为 $1-q$ ，不成功查找时基本操作正好执行 n 次。

$$A(n) = \sum_{I \in D_n} P(I) * T(I)$$

$$= \sum_{i=0}^n P(I) * T(I) = \sum_{i=0}^{n-1} \frac{q}{n} (i+1) + (1-q)n = \frac{(n+1)q}{2} + (1-q)n$$

若 $q=1/2$ ，则 $A(n) = [(n+1)/4] + n/2 \approx 3n/4$ 。



1.1 有关算法的基本概念

- 实现算法2——利用**非降序排列**的特性

$j=1$, 将 x 与 $L[j]$ 比较. 如果 $x=L[j]$, 则算法停止, 输出 j ; 如果 $x > L[j]$, 则把 j 加1, 继续 x 与 $L[j]$ 的比较; 如果 $x < L[j]$, 则停机并输出0. 如果 $j > n$, 则停机并输出 0.

实例

1

2

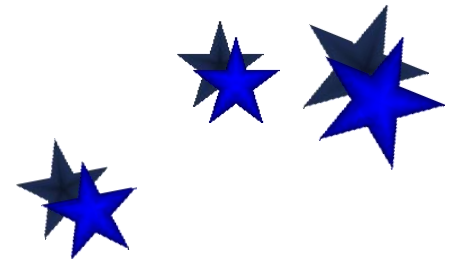
3

4

5

$x = 4$, 需要比较 4 次

$x = 2.5$, 需要比较 3 次



1.1 有关算法的基本概念

```
int Find2(double L[],int n,double x)
{ int i=1;
  while (i<=n)
  { if (L[i]>=x) break;
    i++;
  }
  if (i<=n&&L[i]==x) return i;
  else return 0;
}
```



1.1 有关算法的基本概念

检索算法2的时间估计

最好情况下: $x = L[1], x < L[1]$

$$G(n) = 1$$

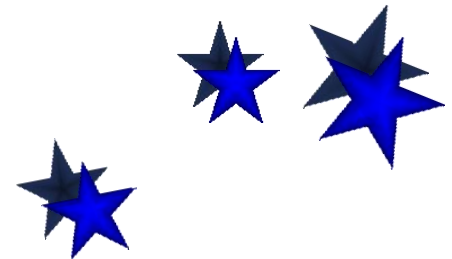
最坏情况下: $x = L[n], L[n-1] < x < L[n], L[n] < x$

$$W(n) = n$$

平均情况下: 输入实例的概率分布: 假设 x 在 L 中每个位置 (成功) 与空隙 (不成功) 的概率都相等



改进检索算法平均时间复杂度是多少?



1.1 有关算法的基本概念

小结:

- 算法最好最坏和平均情况下的时间复杂度定义
- 如何计算上述时间复杂度

练习:

- 请给出一个效率最高的百钱买百鸡问题的求解方案

问题叙述：“鸡翁一，值钱五；鸡母一，值钱三；鸡雏三，值钱一。百钱买百鸡，问鸡翁、母、雏各几何？”

